Relativistic Simulator: Real-Time Interactive Visualization of the Optical Effects of Special Relativity *

Sergio Roncato Maccari *

* *Faculdade de Engenharia de Computação, Universidade Tecnológica Federal do Paraná, PR, Brazil (e-mail: sergiomaccari@alunos.utfpr.edu.br).*

Abstract: Special Relativity is usually taught through isolated formulas, and students rarely develop a visual intuition of how the world would actually look at speeds close to the speed of light. This work presents an interactive simulator, running in real time in the web browser (TypeScript + Three.js/WebGL), that turns the canonical equations of Special Relativity into a navigable 3D scene: Lorentz contraction, time dilation, relativistic aberration (via the past light cone), the relativistic Doppler effect and relativistic beaming, with color computed by spectral reconstruction based on CIE response curves. All the per-frame visible physics is evaluated per vertex on the GPU, in GLSL shaders, and the spectral-reconstruction integral is solved once on the CPU at start-up, which keeps the simulation interactive even on modest hardware. Three observation modes (orbital laboratory, observer's eye and first person at constant velocity) let the user separate what is *measured* from what is *seen*, which is the central conceptual distinction of relativistic optics.

Keywords: Special Relativity; Scientific Visualization; WebGL; Doppler Effect; Computer Graphics.

1. INTRODUCTION

Special Relativity, formulated by Einstein (1905), is presented in undergraduate courses mainly in the form of equations: length contraction, time dilation and the Doppler effect appear as isolated formulas, verified in numerical exercises. Students rarely form a coherent mental picture of *what the world would look like* to a relativistic traveler, partly because no everyday experience comes close to the regime $v \sim c$, partly because the effects never occur in isolation: contraction, light propagation delay, aberration, color shift and brightness variation are simultaneous manifestations of a single geometric structure, Minkowski spacetime.

There is also a recurring conceptual confusion in the introductory study of Special Relativity: treating *measuring* and *seeing* as synonyms. Lorentz contraction is a fact of *measurement* (the simultaneous recording, in the observer's frame, of the two ends of an object); the *image* captured by an eye or a camera, on the other hand, combines past events, because light has a finite speed. Terrell (1959) and Penrose (1959) showed, independently, that a fast object of small angular size does not look flattened in a photograph, but rotated. Distinguishing these two concepts requires visualizing the two stages separately, which is unfeasible with static figures.

Attacking this gap through interactive visualization is not a new idea, and the most influential reference comes from the MIT Game Lab. The game *A Slower Speed of Light* (Kortemeyer et al., 2013) places the player in a world where the speed of light drops progressively down to walking pace, and *OpenRelativity* (MIT Game Lab, 2013; Sherin et al., 2016) generalizes that engine into an open-source toolkit for the Unity engine. These projects established what is now the canonical visual repertoire of the field, that is, contraction, aberration, Doppler and beaming applied to a navigable scene, and they are the acknowledged inspiration for this work. The difference in scope lies in the execution medium and in the intended audience: OpenRelativity presupposes a Unity installation and the compilation of a project, whereas the simulator described here was written from scratch as a static web page, in TypeScript and GLSL on top of WebGL, which opens in any modern browser with no installation and no dedicated hardware. The design priority was to lower as far as possible the barrier to entry for the undergraduate student who simply wants to see the phenomenon.

This work presents an interactive simulator that fills this gap. It is a single-page web application, written in TypeScript on top of the Three.js library (WebGL), in which all the relativistic physics is evaluated *per vertex* on the graphics card, in GLSL shaders. The user adjusts the fraction of the speed of light, $\beta = v/c$, and immediately observes the combined effect (or each effect in isolation, since every effect can be switched on and off individually) of Lorentz contraction, retarded time (apparent position), the relativistic Doppler effect and beaming on the same three-dimensional scene.

The physical scope was restricted to Special Relativity in the simple regime, consistent with the Physics 4 syllabus: sources and observer move with *constant velocity*, without relativistic acceleration and without General Relativity effects. This restriction is sufficient to correctly reproduce the most striking visual phenomena of high-speed motion, and the corresponding theoretical background (Nussenzveig, 2014; Griffiths, 2017) is developed in Section 2. Section 3 describes the implementation methodology (architecture, GPU algorithms

* Work developed in the Physics 4 course. A versão em português deste artigo está disponível no mesmo repositório (`docs/artigo.pdf`).

and observation modes), Section 4 presents the results and the limitations of the model, and Section 5 concludes.

2. THEORETICAL BACKGROUND

2.1 Postulates and the Lorentz transformation

Special Relativity rests on two postulates (Einstein, 1905): (i) the laws of physics have the same form in every inertial frame; (ii) light propagates in vacuum with the same speed c in every inertial frame, regardless of the motion of the source or of the observer. The second postulate, imposed by the experimental evidence of Michelson and Morley (1887), breaks with Galilean kinematics and forces a revision of the concepts of space, time and simultaneity.

For two inertial frames S and S', with S' moving with velocity v along the x axis of S and origins coinciding at $t = t' = 0$, the postulates lead to the *Lorentz transformation*:

$$\begin{aligned} t' &= \gamma\left(t - \frac{v\,x}{c^2}\right), & x' &= \gamma\,(x - v\,t), \\ y' &= y, & z' &= z, \end{aligned} \tag{1}$$

with the central dimensionless quantities of the whole theory:

$$\beta = \frac{v}{c} \in [0, 1), \qquad \gamma = \frac{1}{\sqrt{1-\beta^2}} \geq 1. \tag{2}$$

The transformation preserves the invariant interval $s^2 = c^2t^2 - x^2 - y^2 - z^2$, which guarantees that c is the same for all observers. The simulator always works with the dimensionless velocity $\vec{\beta} = \vec{v}/c$, so the geometry never depends explicitly on c; the numerical value of c only reappears when converting intervals to seconds.

2.2 Length contraction and time dilation

The two classic kinematic effects follow from Eq. (1). A ruler of proper length L_0, at rest in S' and aligned with the motion, has its two ends recorded *at the same instant* by an observer in S; the measured length is

$$L = \frac{L_0}{\gamma} = L_0\sqrt{1-\beta^2}. \tag{3}$$

A clock at rest in S', marking two events separated by the proper time $\Delta\tau$, is measured in S with the dilated interval

$$\Delta t = \gamma\,\Delta\tau \geq \Delta\tau. \tag{4}$$

Both effects are displayed directly by the simulator: contraction is applied to the geometry per vertex; dilation is shown by two analog clocks in the readout panel, with $d\tau = dt/\gamma$ integrated every frame.

2.3 Relativistic velocity addition

If a particle has velocity $\vec{u}$ (in units of c) in a frame that, in turn, moves with $\vec{v}$ relative to the laboratory, the composed velocity is not the Galilean sum. Decomposing $\vec{u}$ into components parallel and perpendicular to $\vec{v}$,

$$\vec{u}\,' = \frac{\vec{u}_\parallel + \vec{v} + \vec{u}_\perp/\gamma_v}{1 + \vec{u}\cdot\vec{v}}, \qquad \gamma_v = \frac{1}{\sqrt{1-|\vec{v}|^2}}. \tag{5}$$

Eq. (5) recovers the Galilean limit when $|\vec{u}|, |\vec{v}| \ll 1$ and saturates at c: for collinear motion, $u' = (u+v)/(1+uv)$, so that $u = 1$ (light) implies $u' = 1$ for any v; the second postulate is respected automatically. The factor $1/\gamma_v$ in the transverse component is the geometric origin of the aberration of light. This equation is implemented in the physics core of the simulator and replicated in GLSL in the vertex shader, where it is evaluated vertex by vertex to compose the velocity of each object with that of the observer, guaranteeing $|\vec{\beta}| < 1$ for any combination.

2.4 Measuring is not seeing: the past light cone

Measuring is an idealized, simultaneous operation in the observer's frame; it is what the Lorentz transformation describes. *Seeing*, on the other hand, is capturing at a single point, the eye, all the light rays arriving now. Since light has a finite speed, the photons reaching the eye at this instant left different points of the source at *different instants in the past*. The image is not an instantaneous portrait, but a stitching of events on the past light cone of the observation event.

Consider the observer at the origin, at time $t = 0$, and a source point with position $\vec{r}$ (at $t = 0$) and constant velocity $\vec{u}$ (in units of c). The past instant $\tau \leq 0$ at which this point emitted the light arriving now satisfies the *retarded-time condition* $|\vec{r} + \vec{u}\,\tau| = -\tau$, which squared gives the quadratic equation

$$\left(|\vec{u}|^2 - 1\right)\tau^2 + 2\,(\vec{r}\cdot\vec{u})\,\tau + |\vec{r}|^2 = 0. \tag{6}$$

Of the two real roots, the physical solution is the *retarded* root ($\tau \leq 0$, on the past light cone); the other would correspond to the future cone, meaningless for an observation. The *apparent position* of the point, that is, the direction from which the light actually arrives, is

$$\vec{r}_{\rm ap} = \vec{r} + \vec{u}\,\tau_{\rm ret}. \tag{7}$$

The simulator solves Eq. (6) for *every vertex* of the scene, every frame. Table 1 summarizes the conceptual distinction that organizes the whole implementation.

2.5 Aberration and the Terrell–Penrose rotation

Aberration is the change in the apparent direction of a light ray when changing frames. Applying Eq. (5) to a light ray yields the closed form

$$\cos\theta' = \frac{\cos\theta + \beta}{1 + \beta\cos\theta}, \tag{8}$$

where θ and θ' are the angles between the line of sight and the direction of motion in the two frames. For $\beta \to 1$, almost the entire celestial sphere is compressed

Table 1. Measuring versus seeing: the central distinction of relativistic optics and its correspondence with the stages of the simulator.

Aspect	Measure (Lorentz)	See (light cone)
Operation	simultaneous in the frame	capture at a point
Length	contraction $1/\gamma$	apparent rotation
Direction	— (no effect)	aberration
Frequency	— (local measure)	Doppler D
Brightness	— (no effect)	beaming D^3
In the shader	vertex boost	τ_{ret}, D, color

forward: the whole sky seems to tip toward the direction of motion. In the simulator, Eq. (8) is not written explicitly: it *emerges* from the combination of the Lorentz boost with the displacement to the apparent position of Eq. (7).

The same geometry explains the result of Terrell (1959) and Penrose (1959): the light coming from the farthest face of an object left earlier, when the object was somewhere else; the observer simultaneously sees the front face and a portion of the face that should be geometrically hidden, exactly as they would see an object at rest, but rotated by the aberration angle of the line of sight; when the object is seen in the direction perpendicular to the motion, this angle α satisfies $\sin\alpha = \beta$. The camera does not photograph the contraction; it photographs a rotation. For extended, nearby objects there are additional deformations and bending of straight lines, which the simulator naturally displays by applying Eq. (6) vertex by vertex (Figure 4).

2.6 Relativistic Doppler effect

Motion also changes the frequency of the received light. With $\hat{n}$ the unit vector pointing from the observer to the source and $\vec{\beta}$ the velocity of the source as measured in the observer's frame, the relativistic *Doppler factor* is

$$D = \frac{f_{\text{obs}}}{f_{\text{emit}}} = \frac{1}{\gamma\left(1 + \vec{\beta}\cdot\hat{n}\right)}, \tag{9}$$

a result of the Lorentz transformation of the wave four-vector: the factor γ is time dilation and the term $1 + \vec{\beta}\cdot\hat{n}$ is the geometric delay/advance. The limiting cases are instructive. For longitudinal approach, $D = \sqrt{(1+\beta)/(1-\beta)} > 1$ (blueshift); for recession, $D = \sqrt{(1-\beta)/(1+\beta)} < 1$ (redshift); and in the transverse direction, $D = 1/\gamma < 1$, the so-called *transverse Doppler effect*, a purely relativistic effect with no classical analogue and a direct consequence of time dilation. The observed wavelength scales as $\lambda_{\text{obs}} = \lambda_{\text{emit}}/D$.

2.7 Relativistic beaming (headlight effect)

The specific intensity I_ν/ν^3 is a Lorentz invariant; it follows that the observed specific intensity transforms as

$$I_{\nu,\text{obs}}(\nu_{\text{obs}}) = D^3\, I_{\nu,\text{emit}}(\nu_{\text{emit}}), \tag{10}$$

with corresponding frequencies related by $\nu_{\text{obs}} = D\,\nu_{\text{emit}}$. As for the origin of the exponent, one factor of D comes from the increase in the energy of each photon, one from the photon arrival rate and D^2 from the angular concentration imposed by aberration, with D^{-1} from the widening of the frequency band completing the balance; the frequency-integrated intensity scales as D^4. The visual consequence is the *headlight effect*: whatever lies ahead of the motion becomes intensely brighter and bluer; whatever lies behind darkens and reddens.

2.8 Color by spectral reconstruction

Applying the Doppler shift to an RGB color requires shifting the light *spectrum* and measuring the eye's response again, not merely changing the hue on the color wheel. The simulator models the emission spectrum as the sum of three *unit-area* Gaussians, one per sRGB primary, centered at the dominant wavelengths of those primaries (611.4, 549.1 and 464.2 nm) and with a full width at half maximum of 40 nm, the order of magnitude of the spectral width of a real display emitter.

Under a shift $s = \lambda_{\text{obs}}/\lambda_{\text{emit}} = 1/D$, photon number conservation implies $f_{\text{obs}}(\lambda) = f_{\text{emit}}(\lambda/s)/s$, so a Gaussian remains a Gaussian with its center and width scaled by s:

$$\mathcal{N}(\lambda;\, \mu,\, \sigma) \;\longmapsto\; \mathcal{N}(\lambda;\, s\mu,\, s\sigma). \tag{11}$$

Intensity does not enter here; that is the beaming factor D^3, applied separately. The observed XYZ is then the integral of the shifted spectrum against the eye response curves $\bar{x}, \bar{y}, \bar{z}$ (CIE 1931, 2°), for which we adopt the multi-lobe *skewed* Gaussian fit of Wyman et al. (2013).

Since the whole path RGB → spectrum → shift → CIE → XYZ → RGB is linear in the rest RGB, it collapses, for each s, into a single 3×3 matrix:

$$\mathbf{C}(s) = \mathbf{M}_{\text{XYZ}\to\text{RGB}}\, \mathbf{P}(s)\, \left[\mathbf{M}_{\text{XYZ}\to\text{RGB}}\, \mathbf{P}(1)\right]^{-1}, \tag{12}$$

where the columns of $\mathbf{P}(s)$ are the XYZ of each shifted primary. The skewed fit has no elementary integral against a Gaussian, so the integration is performed numerically on the CPU, *once*, at start-up, for 512 values of s spaced logarithmically over $[1/200, 200]$. The resulting matrices are stored in a texture, and the shader performs only one interpolation and one matrix-vector product per vertex, with no exponentials at all.

The normalization in Eq. (12) guarantees $\mathbf{C}(1) = \mathbf{I}$ by construction (verified numerically to a maximum error of 2.2×10^{-16}): at rest the displayed color is *exactly* the rest color, with no need for any corrective interpolation between the original and the reconstructed color.

A physical consequence of the model: it does *not reserve* energy in the infrared or ultraviolet. At high β, the three visible Gaussians can be pushed outside the perceptible band and the object darkens. This behavior is consistent for a band-limited detector such as the eye, although a real broad-spectrum source would replenish part of the brightness with radiation coming from outside the visible range (see Section 4.3).

Table 2. Technologies used in the project.

Technology	Role
TypeScript	Application logic with static types: state, modes, controllers and interface.
Three.js	3D scene on WebGL: perspective camera, geometries, rendering loop.
GLSL	Hand-written vertex and fragment shaders; all the visible relativistic physics.
Vite	Development server and production bundler.
lil-gui	Control panel (speed, effects, modes).
GitHub Actions and Pages	Automatic publication of the static site on every push.

3. METHODOLOGY

The development was structured in three layers: the physics *core*, the simulation *engine* and the *interface*, with the visible physics executed entirely on the GPU. The solution is a static web application: there is no physics server; all computation runs in the user's browser.

3.1 Technology stack

Table 2 lists the technologies used and their roles.

3.2 Architecture and data flow

The code separates three responsibilities: `core/` (Special Relativity physics, with $\gamma(\beta)$ and Minkowski velocity addition, plus predefined scenes), `engine/` (orchestration: rendering loop, observation modes, material factory) and `ui/` (control panel and readout HUD). Every frame, the CPU computes the observer's $\vec{\beta}$ vector according to the active mode and hands it to the GPU through *uniforms* (shader input variables) shared by reference among all the materials in the scene: a single write per frame updates all objects simultaneously, without sweeping the material list. Figure 1 summarizes the flow.

3.3 The per-vertex algorithm

The vertex shader executes, for each vertex, four steps in sequence.

Step 1: Lorentz boost and contraction. The vertex position is translated to a frame with the observer at the origin. Next, the position of the object *at the observer's present instant* is sought, that is, at $t' = 0$ of the frame moving with $\vec{\beta}$. Imposing $t' = 0$ on the time transformation of Eq. (1) applied to the vertex world line gives the laboratory instant

$$T_p = \frac{\vec{\beta} \cdot \vec{x}_{\text{lab}}}{1 - \vec{\beta} \cdot \vec{\beta}_{\text{obj}}}, \tag{13}$$

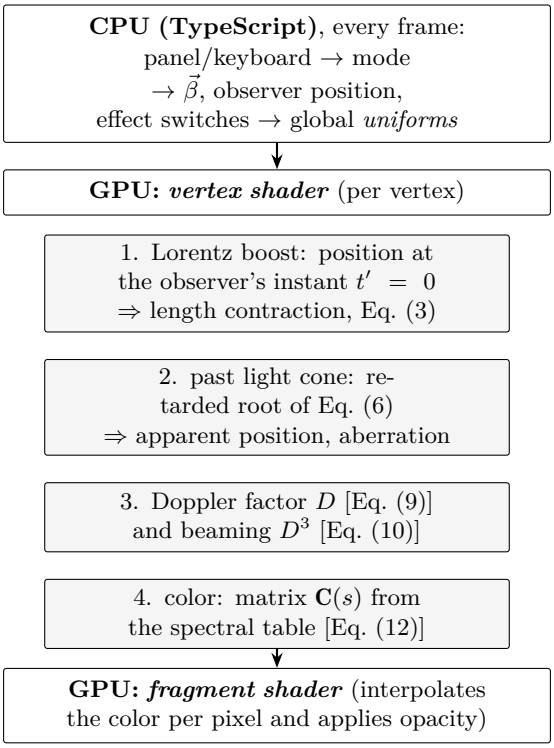

Figure 1. Pipeline of one frame: the CPU prepares a few values per frame and the GPU recomputes the apparent position and the color of every vertex of every object.

and the spatial boost, decomposed along directions parallel and perpendicular to $\hat{\beta}$, leads to the position

$$\begin{aligned} \vec{r}_0 &= \vec{X} + (\gamma - 1)\,(\hat{\beta} \cdot \vec{X})\,\hat{\beta} - \gamma\,\vec{\beta}\,T_p, \\ \vec{X} &= \vec{x}_{\text{lab}} + \vec{\beta}_{\text{obj}}\,T_p. \end{aligned} \tag{14}$$

The contraction arises from the simultaneity slice: since T_p varies from vertex to vertex (relativity of simultaneity), the combination of the boost with the evaluation of each vertex at its own T_p reduces the parallel coordinate of an object at rest in the laboratory to $x_\parallel/\gamma$, which is exactly the contraction of Eq. (3) appearing on screen. The velocity of the object as seen by the observer is composed with Eq. (5).

Step 2: apparent position. With the aberration effect enabled, the shader solves the quadratic of Eq. (6) and selects the physical root $\tau \leq 0$; for $|\vec{u}| < 1$ the two roots always have opposite signs and the choice is unambiguous. The vertex is displaced to the apparent position of Eq. (7); numerical guards handle the nearly linear case ($|\vec{u}|^2 \approx 1$, where the quadratic degenerates).

Step 3: Doppler and beaming. The line of sight $\hat{n} = \vec{r}_{\text{ap}}/|\vec{r}_{\text{ap}}|$ is defined and the factor D of Eq. (9) is computed with the velocity of the object relative to the observer (composed in Step 1) and the corresponding γ. Brightness is multiplied by D^3 when beaming is enabled.

Step 4: color. The rest color goes through the spectral reconstruction of Section 2.8, with the shift $\lambda_{\text{obs}} =$

λ_{emit}/D, and the result is interpolated by rasterization down to the fragment shader, which is limited to applying opacity.

Each effect (contraction, aberration, Doppler, beaming) is controlled by its own switch in the panel, passed to the shader as a 0/1 uniform (a linear mix in the case of contraction, a conditional branch for the others). This way, the student can enable one effect at a time and identify the contribution of each term of the equations.

3.4 Computational cost: per vertex versus per pixel

The spectral reconstruction is a numerical integral over the visible band, far too expensive to evaluate in real time. The engineering choice was to move it entirely out of the rendering loop: since Eq. (12) shows that the result depends on a single scalar parameter s, the integral is solved on the CPU at start-up and tabulated. What remains in the shader is a texture lookup and a matrix-vector product, with no exponentials.

The second choice is *where* to pay the residual cost: per *pixel* (millions of evaluations per frame at full screen) or per *vertex* (typically orders of magnitude fewer evaluations), with the graphics card linearly interpolating the result along each triangle, in the style of Gouraud shading. Per-vertex computation was adopted, decisive for keeping interactivity on modest GPUs. The price is the possibility of interpolation artifacts on coarse meshes; the mitigation is geometric: the scene meshes are subdivided enough that the color variation between neighboring vertices is small. In addition, a quality selector adjusts the renderer's device pixel ratio, reducing the number of rasterized fragments on slow machines.

3.5 Observation modes

Three modes, selectable in the panel, define how the $\vec{\beta}$ vector and the camera are built every frame:

1. **Laboratory (orbital):** external orbital camera; the conceptual observer is at the origin and $\vec{\beta}$ is assembled from a magnitude slider and a chosen axis. It is the ideal mode to see, from the outside, the contraction and the apparent deformation of the scene. Presets fix $\gamma = 2$, 5 and 10 via $\beta = \sqrt{1 - 1/\gamma^2}$.
2. **Observer's eye:** the observer stays still at the origin and $\vec{\beta}$ *follows the gaze direction*: turning the eye rotates the direction of motion and, with it, the blue-shifted region (ahead) and the red-shifted region (behind).
3. **First person:** keyboard navigation (WASD) with constant velocity: at every instant the observer occupies a well-defined inertial frame, consistent with the Physics 4 scope, with no relativistic acceleration phase.

In every mode the magnitude is saturated ($|\vec{\beta}| \leq 0.9999$) to avoid $\gamma \to \infty$, and a readout panel (HUD) displays β, γ, $L/L_0 = 1/\gamma$, the dilation factor, the frame rate (FPS) and two analog clocks, one for the laboratory and one for the observer, integrated with $d\tau = dt/\gamma$, which makes time dilation visible in real time. The interface is available in Portuguese and English, selectable in the control panel.

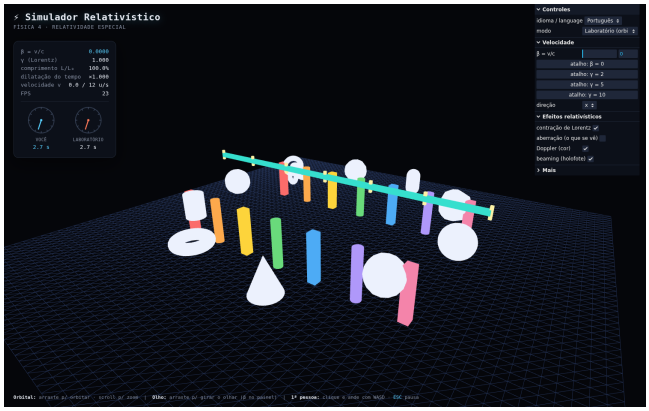

Figure 2. Laboratory scene at rest ($\beta = 0$, $\gamma = 1$): original geometry and colors; the two HUD clocks advance at the same rate.

4. RESULTS

4.1 Observable effects

Figure 2 shows the reference scene at rest ($\beta = 0$): a grid floor, a horizontal ruler and a gallery of colored solids. In it, the two HUD clocks advance together ($\gamma = 1$).

With $\beta = 0.7$ along the x axis (Figure 3), the following appear simultaneously: the contraction of the shapes along the direction of motion ($L/L_0 = 71.4\%$, HUD); the Doppler shift, with objects blue-shifted in the approach direction, red-shifted in the opposite one and the floor displaying the continuous color gradient along the line of sight; beaming, which darkens the recession region; and time dilation on the clocks (3.5 s of proper time against 4.9 s of laboratory time, factor $\times 1.400 = \gamma$).

Enabling the apparent position (Figure 4), the retarded time of Eq. (6) displaces each vertex individually: straight columns appear bent and the solids, rotated. It is the manifestation, for extended objects, of the Terrell–Penrose rotation, and the direct visual difference between *what is measured* (Figure 3) and *what is seen* (Figure 4).

In the Observer's Eye mode (Figure 5), the observer standing at the origin "looks" along their own direction of motion: even at $\beta = 0.25$ the blueshift ahead is clear, with the color gradient sweeping continuously across the scene as the line of sight moves away from the direction of motion.

At $\beta = 0.9$ (Figure 6), the spectral shift is so intense that the visible light of several objects leaves the perceptible band and they darken, a predicted consequence of the color model without spectral reserve (Section 2.8):

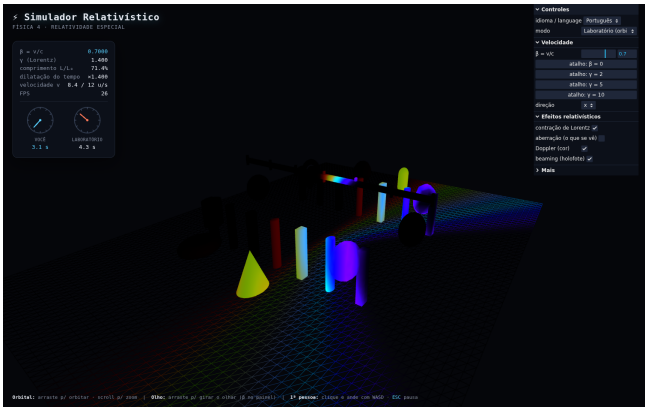

Figure 3. Same scene with $\beta = 0.7$: contraction ($L/L_0 = 71.4\%$), Doppler and beaming; the clocks display the dilation $\gamma = 1.4$.

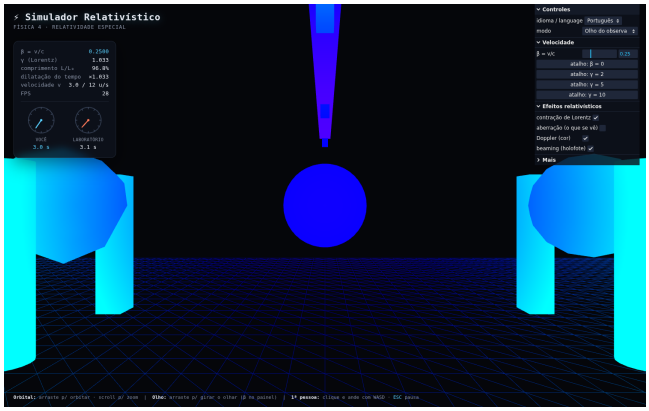

Figure 5. Observer's Eye mode, $\beta = 0.25$ along the gaze direction: blueshift ahead, with a continuous color gradient across the scene.

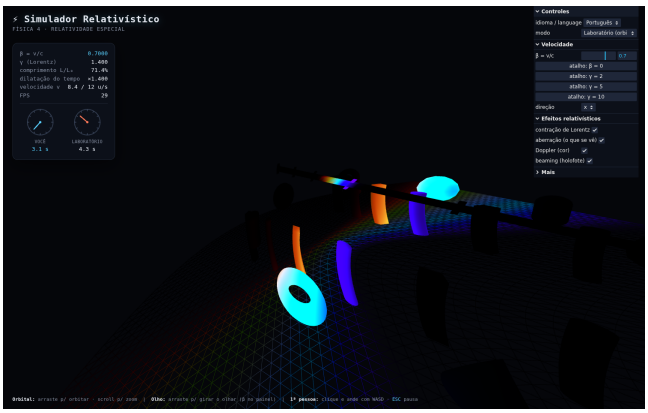

Figure 4. $\beta = 0.7$ with apparent position (past light cone) enabled: straight columns appear bent and the solids rotated (Terrell–Penrose).

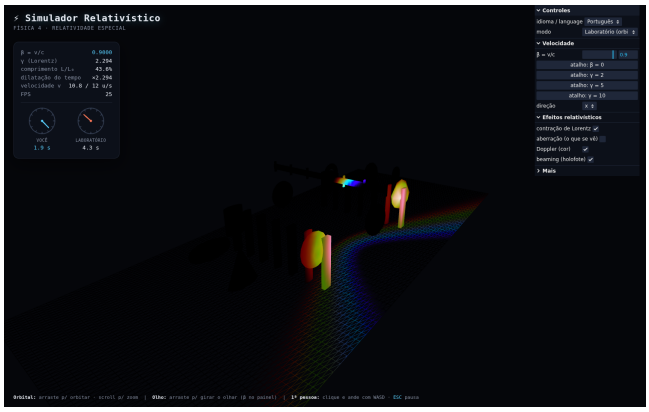

Figure 6. $\beta = 0.9$ ($\gamma = 2.294$): objects darken when their visible light is shifted outside the perceptible band, the expected behavior of the band-limited color model.

the intense blue disappears into the ultraviolet ahead and the red, into the infrared behind.

4.2 Performance

By concentrating the physics in the vertex shader and keeping the fragment shader trivial, the simulation sustains interactive rates (tens of frames per second) even under software rendering, without a dedicated GPU; the figure captures, taken under those conditions, register between 23 and 28 FPS on the HUD. On common hardware with graphics acceleration, the rate is limited by the display refresh. The quality selector reduces the renderer's pixel ratio when needed, and the HUD framerate meter lets the user calibrate the adjustment.

4.3 Limitations and simplifications

Table 3 consolidates the adopted simplifications, their nature and the main effect of each one. Two of them deserve comment. The color model does not reserve energy in the infrared/ultraviolet: at high β, objects darken instead of displaying the brightness that a real broad-spectrum source would keep, although the qualitative behavior of the shift (blueing/reddening) remains correct. In addition, the scope is restricted to constant velocity: the transformations applied at each frame are rigorously correct for the instantaneous inertial frame, but velocity changes are idealized, with no continuous acceleration phase and none of the associated phenomena, such as Thomas precession, a restriction aligned with the Physics 4 level.

4.4 Availability

The simulator is published and can be run in any modern browser, with no installation:

`https://sergiomaccari.github.io/Simulador-Relativ-stico/`

The complete source code (TypeScript, GLSL and this document) is available at:

`https://github.com/sergiomaccari/Simulador-Relativ-stico`

For local execution, `npm install` and `npm run dev` at the repository root are enough.

5. CONCLUSION

The developed simulator demonstrates that the optical effects of Special Relativity (Lorentz contraction,

Table 3. Main simplifications of the model and their effects.

Simplification	Nature	Main effect
Color without IR/UV reserve	color model	darkening at high β
No lighting/shadows	rendering	self-luminous appearance
Constant velocity	scope (SR)	no relativistic acceleration
Per-vertex color (Gouraud)	performance	artifacts on coarse meshes
Arbitrary units, adjustable c	didactics	only ratios (β, γ, D) faithful

time dilation, aberration via the past light cone, the relativistic Doppler effect and beaming) can be visualized in an interactive, quantitative and physically grounded way in an open web application, with no installation. The per-vertex GPU implementation proved sufficient to keep interactivity even without dedicated graphics acceleration. The individual switches for each effect, together with the explicit distinction between *measuring* (Lorentz transformation) and *seeing* (retarded time), give the student a visual laboratory in which every term of the canonical equations has an observable counterpart on screen. Natural extensions include the spectral reserve in the infrared/ultraviolet in the color model, per-fragment Doppler computation and an advanced mode with accelerated (hyperbolic) motion, the latter already outside the scope of Physics 4.

REFERENCES

Einstein, A. (1905). Zur Elektrodynamik bewegter Körper. *Annalen der Physik*, 17, 891–921.

Griffiths, D.J. (2017). *Introduction to Electrodynamics*. 4th ed. Cambridge University Press, Cambridge.

Kortemeyer, G., Tan, P. and Schirra, S. (2013). A Slower Speed of Light: developing intuition about special relativity with games. In *Proceedings of the 8th International Conference on the Foundations of Digital Games (FDG 2013)*, 400–402. SASDG, Chania, Greece.

Michelson, A.A. and Morley, E.W. (1887). On the relative motion of the Earth and the luminiferous ether. *American Journal of Science*, 34, 333–345.

MIT Game Lab (2013). *OpenRelativity* [computer software]. Massachusetts Institute of Technology. MIT License. `https://github.com/MITGameLab/OpenRelativity`.

Nussenzveig, H.M. (2014). *Curso de Física Básica, vol. 4: Ótica, Relatividade, Física Quântica*. 2nd ed. Blucher, São Paulo.

Penrose, R. (1959). The apparent shape of a relativistically moving sphere. *Proceedings of the Cambridge Philosophical Society*, 55(1), 137–139.

Sherin, Z.W., Cheu, R., Tan, P. and Kortemeyer, G. (2016). Visualizing relativity: the OpenRelativity project. *American Journal of Physics*, 84(5), 369–374.

Terrell, J. (1959). Invisibility of the Lorentz contraction. *Physical Review*, 116(4), 1041–1045.

Wyman, C., Sloan, P.-P. and Shirley, P. (2013). Simple analytic approximations to the CIE XYZ color matching functions. *Journal of Computer Graphics Techniques*, 2(2), 1–11.